



Desafio SBPO 2025 – Descrição do Problema

O Problema da Seleção de Pedidos Ótima

Janeiro de 2025

Conteúdo

1	Introdução	2
2	Descrição do problema	2
2.1	Definições e notações	2
2.2	Conjuntos	4
2.3	Constantes	4
2.4	Objetivo do problema e restrições	4
3	Exemplo	5

1 Introdução

A jornada de um pacote dentro da rede de entregas do MERCADO LIVRE começa muito antes da fase de transporte. Uma vez que um pedido é feito por um cliente, ele entra em um *backlog* de pedidos a coletar e recebe um prazo de processamento com base no tempo de entrega prometido. A próxima etapa envolve a coleta no armazém dos itens que compõem o pedido para posterior embalagem e transporte. Obviamente, a coleta de itens de um pedido individualmente é menos eficiente do que a coleta realizada ao agrupar-se subconjuntos de pedidos, chamados de *waves* de pedidos ou, simplesmente, *waves*. Isso porque, a coleta simultânea de pedidos permite aproveitar as sinergias entre os locais de coleta dos itens dos diferentes pedidos. O processo de selecionar sucessivamente do *backlog* o próximo subconjunto de pedidos a serem coletados, por meio de uma ou várias rotas, é conhecido como **coleta de pedidos em waves**. Entretanto, para maximizar a produtividade da operação de coleta, a seleção dos pedidos que irão compor a próxima *wave* deve ser feita de forma criteriosa. Assim, o foco deste desafio é propor algoritmos para resolver o problema de encontrar uma *wave* de pedidos ótima, maximizando a produtividade da coleta.

2 Descrição do problema

Nesta seção, descreveremos o problema em detalhes.

2.1 Definições e notações

Começamos dando definições e notações básicas.

- **Item:** um produto que pode ser solicitado.
- **Pedido:** uma lista específica de itens solicitados por um cliente junto com suas quantidades.
- **Coleta:** o processo de recuperar produtos do armazém.
- **Backlog:** o conjunto de pedidos cujos itens ainda não foram coletados.
- **Wave:** um subconjunto de pedidos do *backlog* selecionados para serem processados (ou roteados) juntos, com o objetivo de maximizar a produtividade das operações de coleta.
- **Corredor:** para fins deste desafio, um armazém pode ser concebido como um único piso retangular contendo fileiras paralelas de estantes compostas múltiplas camadas de prateleiras, cada uma delas com um *endereço* associado. Nestas prateleiras são armazenados os itens que podem fazer parte de um pedido. Os espaços livres entre duas fileiras de estantes consecutivas são chamados de *corredores* e permitem a circulação de pessoas ou equipamentos que fazem a coleta dos itens presentes em um pedido.

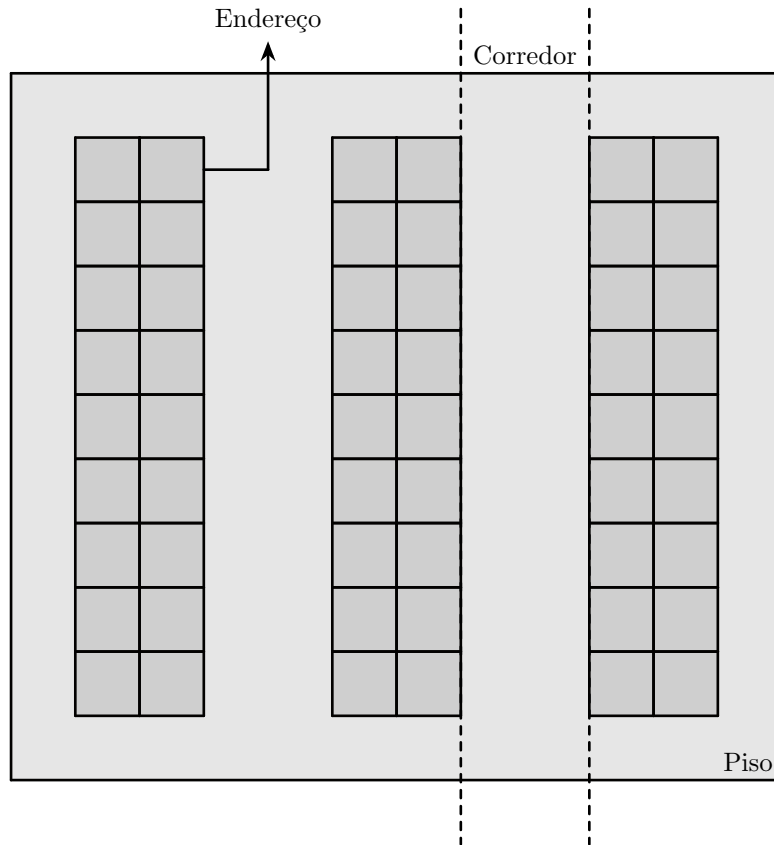


Figura 1: Layout do piso de um armazém com seus elementos básicos.

Na Figura 1 vê-se um exemplo de um diagrama representando o piso de armazém.

Abaixo estão os conjuntos e constantes necessárias para definir o problema-alvo deste desafio.

2.2 Conjuntos

- $\mathcal{O}$ Conjunto de pedidos no *backlog*.
- $\mathcal{I}_o$ Subconjunto de itens solicitados pelo pedido $o \in \mathcal{O}$.
- $\mathcal{I}$ Conjunto de itens, onde $\mathcal{I} = \bigcup_{o \in \mathcal{O}} \mathcal{I}_o$.
- $\mathcal{A}_i$ Subconjunto de corredores contendo pelo menos uma unidade do item i .
- $\mathcal{A}$ Conjunto de corredores, onde $\mathcal{A} = \bigcup_{i \in \mathcal{I}} \mathcal{A}_i$.

2.3 Constantes

- u_{oi} Número de unidades do item $i \in \mathcal{I}$ solicitado pelo pedido $o \in \mathcal{O}$.
- u_{ai} Número de unidades do item $i \in \mathcal{I}$ disponíveis no corredor $a \in \mathcal{A}$.
- LB / UB Limite inferior / superior do *tamanho* da *wave* (definido abaixo).

2.4 Objetivo do problema e restrições

O objetivo do problema de coleta de pedidos em *waves* é selecionar um subconjunto de pedidos do *backlog*, ou seja, a *wave*, e um subconjunto de corredores (loais de coleta) que maximizem a produtividade do processo de coleta. Isso é conseguido ao criar condições favoráveis à geração de rotas de coleta eficientes. Para este desafio, vamos explorar uma dessas condições, que é concentrar os itens da *wave* na menor quantidade possível de corredores, respeitando as restrições descritas abaixo.

De maneira mais formal, seja $O' \subset \mathcal{O}$ o subconjunto de pedidos na *wave*, e $A' \subset \mathcal{A}$ o subconjunto de corredores que serão visitados para coletar todos os itens em O' . O objetivo é maximizar o número de itens coletados por corredor visitado, matematicamente representado por:

$$\max \frac{\sum_{o \in O'} \sum_{i \in \mathcal{I}_o} u_{oi}}{|A'|} \quad (1)$$

Existem essencialmente três famílias de restrições. As duas primeiras estabelecem limites operacionais no número total de unidades que precisam ser coletadas para os pedidos incluídos na *wave*. Elas são expressas como:

$$\sum_{o \in O'} \sum_{i \in \mathcal{I}_o} u_{oi} \geq \text{LB}, \quad (2)$$

$$\sum_{o \in O'} \sum_{i \in \mathcal{I}_o} u_{oi} \leq \text{UB}. \quad (3)$$

A terceira família de restrições assegura que os corredores selecionados tenham armazenamento suficiente para cada um dos itens incluídos nos pedidos dentro da *wave*. Isso se traduz em:

$$\sum_{o \in O'} u_{oi} \leq \sum_{a \in A'} u_{ai}, \quad \forall i \in I_o \text{ com } o \in O'. \quad (4)$$

Qualquer par (O', A') de $\mathcal{O} \times \mathcal{A}$ que satisfaça às restrições (2), (3) e (4) constitui uma *solução viável* para o problema de coleta de pedidos em *wave*. Neste contexto, a *wave* O' também é considerada *viável*.

No entanto, observe que pode haver múltiplos candidatos para o subconjunto A' de corredores selecionados que satisfaçam a restrição (4) para a *wave* dada. Naturalmente, o objetivo é identificar entre esses subconjuntos um que maximize a função objetivo (1).

3 Exemplo

Considere o seguinte exemplo com 5 pedidos, 5 itens diferentes e 5 corredores. O número de unidades de cada item solicitado por cada pedido no *backlog* é dado na Tabela 1, enquanto a disponibilidade de itens em cada corredor é dada na Tabela 2. Os limites inferior e superior para o tamanho da *wave* são 5 e 12, respectivamente.

Pedido	Item 0	Item 1	Item 2	Item 3	Item 4
0	3	0	1	0	0
1	0	1	0	1	0
2	0	0	1	0	2
3	1	0	2	1	1
4	0	1	0	0	0

Tabela 1: Número de unidades de cada item solicitado por cada pedido.

Corredor	Item 0	Item 1	Item 2	Item 3	Item 4
0	2	1	1	0	1
1	2	1	2	0	1
2	0	2	0	1	2
3	2	1	0	1	1
4	0	1	2	1	2

Tabela 2: Número de unidades de cada item disponível em cada corredor.

Alguns exemplos de *waves* viáveis, e suas atribuições de corredores, são:

- Pedidos 0 e 4, usando corredores 0 e 1.
 - Total de unidades: 5;
 - Número de corredores: 2;
 - Valor objetivo: 2.5.
- Pedidos 0, 2 e 3, usando corredores 1, 3 e 4.
 - Total de unidades: 12;
 - Número de corredores: 3;
 - Valor objetivo: 4.
- Pedidos 0, 1, 2 e 4, usando corredores 1 e 3.
 - Total de unidades: 10;
 - Número de corredores: 2;
 - Valor objetivo: 5.

Na última solução, o número médio de itens coletados por corredor (o valor objetivo) é 5, maior que os das outras soluções mostradas, tornando a última solução preferível em relação às demais. De fato, pode-se mostrar que esta solução é ótima para a instância dada.

Formato e Descrição dos Arquivos

Para uma melhor compreensão recomenda-se ler primeiro o arquivo de descrição do problema.

Entrada de Dados

A entrada é fornecida em um arquivo de texto com o seguinte formato:

- Primeira linha: contém três inteiros o (número de pedidos), i (itens) e a (corredores).
- Próximas o linhas: cada linha começa com um inteiro k , seguido por k pares de inteiros, representando o número do item e o número de unidades solicitadas pelo pedido. Dentro deste grupo de linhas e, para j em $\{1, \dots, o\}$, a j -ésima linha contém dados relacionados ao pedido indexado por $j - 1$, significando que os pedidos são numerados de 0 a $o - 1$. Além disso, assume-se que os itens estão indexados de 0 a $i - 1$.
- Próximas a linhas: cada linha começa com um inteiro l , seguido por l pares de inteiros, representando o número do item e o número de unidades disponíveis no corredor. Similar ao caso dos pedidos, dentro deste grupo de linhas e, para j em $\{1, \dots, a\}$, a j -ésima linha contém dados relacionados ao corredor indexado por $j - 1$, significando que os corredores são numerados de 0 a $a - 1$.
- Última linha: contém dois inteiros, LB e UB , que são os limites inferior (*lower bound*) e superior (*upper bound*) do tamanho da *wave* (um subconjunto de pedidos), respectivamente. Esses limites são expressos em quantidade de unidades de itens.

Assim, a entrada para a instância dada como exemplo no arquivo de descrição do problema é:

```
5 5 5
2 0 3 2 1
2 1 1 3 1
2 2 1 4 2
4 0 1 2 2 3 1 4 1
1 1 1
4 0 2 1 1 2 1 4 1
4 0 2 1 1 2 2 4 1
3 1 2 3 1 4 2
4 0 2 1 1 3 1 4 1
4 1 1 2 2 3 1 4 2
5 12
```

Saída da Solução

A saída deve ser um arquivo de texto com o seguinte formato:

- Primeira linha: contém um inteiro n , o número de pedidos na *wave* (subconjunto de pedidos) retornada pelo algoritmo.
- Próximas n linhas: cada linha contém um inteiro representando o índice de um pedido da *wave* computada.
- Linha seguinte: contém um inteiro m que representa o número de corredores visitados na *wave* computada.
- Próximas m linhas: cada linha contém um inteiro representando o índice de um corredor visitado pela *wave* computada.

Por exemplo, a saída para da solução do exemplo dado no arquivo de descrição do problema seria:

```
4
0
1
2
4
2
1
3
```

Verificador de Solução

Um script Python, chamado `checker.py`, é fornecido para verificar a viabilidade de uma solução com base nos arquivos de entrada e saída passados como parâmetros, além de calcular o valor objetivo. O script pode ser executado usando o seguinte comando: `python3 checker.py <input-file> <output-file>`